GUÍA MAESTRA DE DEFENSA PRESENCIAL (10 MINUTOS)

SISTEMA DE GESTIÓN DE RESULTADOS (SGR) — ILUSTRE MUNICIPALIDAD DE LA SERENA
Ingeniería de Software (TI2043) · Evaluación Sumativa N° 3 (Ponderación 35%) · INACAP La Serena

Asignatura:	Ingeniería de Software (TI2043)	**Ponderación:**	35% Nota Final
Modalidad:	Grupal con evaluación individual (3-4 alumnos)	**Tiempo Límite:**	10 minutos cronometrados
Despliegue AWS EC2:	http://3.229.11.31/	**Wireframe Interactivo:**	http://3.229.11.31/wireframes/
Repositorio GitHub:	https://github.com/iScreamyy6/Backend	**Arnés Automatizado:**	python test_suite.py (14 suites OK)

1. Alineación Estratégica con la Rúbrica de Evaluación (100 Puntos)

Para asegurar la calificación máxima (Nota 7.0), cada bloque de la exposición cubre directamente uno de los cinco criterios oficiales:

Criterio de Evaluación	Pts	Foco de la Defensa Técnica	Evidencia Concreta a Mostrar
1. Requerimientos RF y RNF en Prototipo	**25**	Cumplimiento del negocio municipal, velocidad y modularidad MVT.	Prototipo en AWS EC2 respondiendo en < 200 ms con Bootstrap 5 y paleta oficial.
2. Aplicación Metodológica (Scrum / Cascada)	**20**	Justificar modelo híbrido: Cascada para análisis y Scrum para desarrollo.	Cronograma de 3 Sprints, artefactos del Product Backlog e historias de usuario.
3. Estándares de Calidad y Gestión	**20**	Versionamiento semántico, documentación y testing automatizado.	Ejecución en vivo de test_suite.py (14 suites pasando 100% limpias).
4. Estándares de Seguridad en la Construcción	**15**	OWASP Top 10, tokens criptográficos, RBAC y Ley 21.459.	Recuperación por PIN hash, tokens SHA-256 temporales y control de accesos por grupo.
5. Reflejo de Diagramas de Caso de Uso	**20**	Correspondencia exacta de los 10 Casos de Uso (CU-01 a CU-10).	Recorrido de los 8 Mockups interactivos en /wireframes/.

2. Cronograma Minuto a Minuto de la Exposición

Minuto	Bloque Temático	Qué Proyectar en Pantalla	Criterio Rúbrica
0:00 - 1:30	**Bloque 1:** Apertura, Problemática y Arquitectura Cloud (IaaS/SaaS)	Portada y Diagrama de Arquitectura AWS EC2 + MySQL	Criterio 1 + 3.1.1
1:30 - 3:00	**Bloque 2:** Metodología Híbrida (Cascada + Scrum) y Sprints	Esquema de 3 Sprints y Backlog en Diapositivas / Docs	Criterio 2 (20 pts)
3:00 - 5:30	**Bloque 3:** Demostración Wireframes y Casos de Uso (CU-01 a CU-10)	Navegador web en vivo en /wireframes/ (8 mockups)	Criterio 5 (20 pts)
5:30 - 7:30	**Bloque 4:** Demostración del Prototipo Real en AWS EC2 y Seguridad	Sistema real en http://3.229.11.31/ (Login, RBAC, Filtros)	Criterio 4 (15 pts)
7:30 - 9:00	**Bloque 5:** Calidad y Arnés de Pruebas Automatizadas en Vivo	Terminal ejecutando python test_suite.py (14 suites)	Criterio 3 (20 pts)
9:00 - 10:00	**Bloque 6:** Conclusiones, Ética Profesional (ACM/IEEE) y Cierre	Diapositiva final con enlaces GitHub, AWS y ronda de preguntas	Criterio 3.1.2

3. Guion Técnico y Speech Sugerido por Bloque

Bloque 1: Apertura, Problemática y Arquitectura Cloud (0:00 – 1:30)

«Buenos días profesor y comisión evaluadora. Hoy presentamos la culminación del Sistema de Gestión de Resultados (SGR) para la Ilustre Municipalidad de La Serena. El problema central que resolvimos fue la dispersión operativa: las delegaciones municipales como Las Compañías o La Antena operaban con planillas desconectadas, duplicando solicitudes y perdiendo el seguimiento de compromisos vecinales. Para resolverlo, diseñamos una arquitectura cloud moderna bajo el paradigma IaaS (Infraestructura como Servicio) sobre una instancia AWS EC2 Ubuntu 24.04, sirviendo el backend con Django 5.x y Python 3.14, y persistiendo en MySQL 8.x. Esta infraestructura permite a la municipalidad evolucionar hacia un modelo SaaS interno centralizado.»

Bloque 2: Metodología Híbrida y Gestión de Sprints (1:30 – 3:00)

«En el criterio metodológico, adoptamos un enfoque Híbrido de Ingeniería de Software rigurosamente justificado: 1. ¿Por qué Modelo Cascada para la Etapa 1? En el sector municipal rigen normativas de auditoría estrictas. Era imperativo levantar formalmente los 38 Requerimientos Funcionales, normalizar el Modelo Entidad-Relación en 3NF y diseñar los Casos de Uso antes de codificar. Improvisar un modelo relacional comprometería la integridad referencial. 2. ¿Por qué Scrum para las Etapas 2 y 3? Una vez establecida la arquitectura base, organizamos el desarrollo en 3 Sprints de 2 semanas, usando historias de usuario, revisiones de sprint y retrospectivas. Esto nos permitió entregar valor funcional continuo: en el Sprint 1 maquetamos prototipos y autenticación; en el Sprint 2 integramos los 6 mantenedores CRUD y MySQL; y en este Sprint 3 completamos seguridad, recuperación criptográfica, el tablero kanban y el despliegue cloud en AWS.»

Bloque 3: Demostración de Wireframes y Casos de Uso CU-01 a CU-10 (3:00 – 5:30)

«A continuación demostramos nuestro artefacto interactivo de wireframes en /wireframes/, con la paleta cromática oficial (#AD0000) e interactividad completa: • Mockup 1 (CU-01/02): Login dual y recuperación de contraseña con PIN de 6 dígitos. • Mockup 2 (CU-09): Dashboard dinámico con simulación RBAC (Administrador, Supervisor y Operador). • Mockup 3 (CU-05): Registro Unificado con autocompletado AJAX por RUT y derivación opcional al Tubo de Trabajo. • Mockup 4 (CU-07): Tablero Kanban de 4 columnas con movimiento interactivo de tarjetas y folios relacionales [GES-XXXXX]. • Mockup 5 (CU-06): Ficha Social y tramos RSH (40%, 60%, 70%) para casos críticos. • Mockup 6: Semáforo de Metas SGR con cálculo de avance dinámico (+5%) y reglas tricolor. • Mockup 7 (CU-03/04/10): Mantenedores CRUD con pestañas (Delegaciones, Usuarios, Roles) y exportación simulada a Excel. • Mockup 8 (CU-08): Bandeja de Evidencias Fotográficas con ciclo de vida dual (al ingreso y posterior en terreno) y validación de supervisor.»

Bloque 4: Demostración del Prototipo Real en AWS EC2 y Seguridad (5:30 – 7:30)

«Pasamos ahora al sistema en producción real desplegado en AWS EC2 (http://3.229.11.31/). No usamos el panel genérico de Django Admin: 1. Autenticación Segura (Ley 21.459 y OWASP A07): Iniciamos con el supervisor 'acuaman'. Django bloquea accesos no autorizados mediante middleware y hash PBKDF2/SHA-256. 2. Recuperación por Correo y Token (CU-02): Envío real vía SMTP Gmail con código PIN y enlace de token temporal de un solo uso. 3. Control de Acceso RBAC (OWASP A01): Si el operador 'flash' intenta ingresar a /mantenedores/usuarios/, la vista lo rechaza con mensaje de advertencia. 4. Filtro de Delegaciones: El usuario puede alternar entre su delegación territorial ([D02] La Antena) o la vista comunal consolidada.»

Bloque 5: Aseguramiento de Calidad y Pruebas Automatizadas (7:30 – 9:00)

«Para asegurar máxima calidad del software, ejecutamos en vivo en la terminal: python test_suite.py. Como observan, la suite ejecuta en segundos 14 baterías de pruebas automatizadas: • Endpoints HTTP 200 de los 6 mantenedores CRUD. • Generación y descarga binaria de planillas .xlsx con openpyxl. • Cascada AJAX entre Tipos y Subatenciones. • Fusión transaccional de gestiones derivadas al Tubo y generación de folio EVI. • Detección automática de compromisos atrasados en el Kanban. • Consumo e invalidación de tokens seguros. Y lo más importante para la calidad del software: la suite es 100% idempotente, limpiando sus registros temporales y garantizando que la base de datos municipal mantiene exactamente sus 24 gestiones y 13 compromisos históricos tanto antes como después.»

Bloque 6: Conclusiones, Ética Profesional (ACM/IEEE) y Cierre (9:00 – 10:00)

«Para concluir, el proyecto se construyó bajo el Código de Ética ACM/IEEE: Principio 1.2 (evitar el daño) mediante la protección de datos sensibles de familias vulnerables bajo la Ley 19.628; y Principio 2.5 (alta calidad técnica) respaldado por nuestra suite automatizada, arquitectura MVT limpia y versionamiento con más de 20 commits semánticos en GitHub. El SGR está disponible en vivo en AWS EC2, con su repositorio documentado y wireframe interactivo listo para operar. Quedamos a disposición del profesor para la ronda de preguntas. Muchas gracias.»

4. Banco de Preguntas del Docente (FAQ / Preguntas Trampa) y Respuestas 7.0

Respuestas preparadas y fundamentadas técnicamente ante las preguntas más probables de la comisión:

Pregunta: «¿Por qué usaron un modelo híbrido Cascada-Scrum y no Scrum puro?»

Respuesta Técnica: En el sector público rigen normativas de auditoría estrictas. Era imperativo levantar formalmente los 38 requerimientos normativos, normalizar el Modelo Entidad-Relación en 3NF y diseñar los diagramas de Casos de Uso previamente (Cascada) para asegurar la integridad referencial. Luego se adoptó Scrum para iterar quincenalmente en pantallas, integración de servicios (SMTP, Excel) y despliegue cloud.

Pregunta: «¿Cómo garantiza el sistema el cumplimiento de la Ley 21.459 sobre Delitos Informáticos?»

Respuesta Técnica: La Ley 21.459 sanciona el acceso ilícito e interceptación. El sistema lo previene con: 1) Control RBAC estricto a nivel de vista (Django decorators); 2) Claves hasheadas con PBKDF2/SHA-256; 3) Tokens de recuperación temporales de un solo uso; 4) Trazabilidad y auditoría completa por usuario y delegación; 5) Consultas parametrizadas en el ORM de Django que impiden inyecciones SQL.

Pregunta: «¿Cómo funciona el ciclo de vida de las evidencias fotográficas?»

Respuesta Técnica: Tiene un ciclo dual: 1) Al momento del ingreso: el funcionario puede adjuntar la foto inicial del requerimiento en el Formulario Unificado; 2) Posterior a la creación: la cuadrilla en terreno accede al detalle de la gestión y sube fotos de avance o término (EVI-YYYYMMDD-XXXX); 3) Supervisión: en /actividades/evidencias/, el supervisor valida, aprueba o rechaza formalmente el trabajo.

Pregunta: «¿Por qué eligieron AWS EC2 (IaaS) en lugar de un PaaS gratuito como Render o PythonAnywhere?»

Respuesta Técnica: Los PaaS gratuitos suspenden instancias por inactividad ('cold starts' de hasta 60s), tienen memoria RAM reducida y no permiten configurar servicios daemon en segundo plano. AWS EC2 provee cómputo dedicado continuo bajo Ubuntu Linux con Apache como proxy inverso, Gunicorn y MySQL 8.x en EBS, garantizando un tiempo de respuesta menor a 200 ms exigido para la atención ciudadana.

Pregunta: «¿Cómo aseguran que test_suite.py sea realmente idempotente?»

Respuesta Técnica: Cada prueba que inserta registros temporales (como ATC-00056 o usuarios de prueba) ejecuta al final una rutina de limpieza explícita en su bloque de teardown. De este modo, la base de datos mantiene exactamente sus 24 gestiones y 13 compromisos históricos antes y después de la ejecución.

5. Checklist Técnico y Logístico (Antes de Subir al Proyector)

Momento	Acciones Críticas de Verificación
30 Min Antes	• Probar conexión a internet. • Abrir navegador en incógnito con 3 pestañas: 1) Producción AWS (http://3.229.11.31/), 2) Wireframes (/wireframes/), 3) Repositorio GitHub. • Abrir terminal con el comando preparado: python test_suite.py.
5 Min Antes	• Conectar HDMI/proyector y ajustar resolución. • Activar cronómetro visible en celular de un integrante (alarmas a los 7 y 9 minutos). • Recordar credenciales rápidas: Admin (admin/admin), Supervisor (acuaman/admin), Operador (flash/admin).
Durante Exposición	• Hablar con terminología formal (MVT, 3NF, idempotencia, RBAC, IaaS, OWASP, Ley 21.459). • Si internet falla: abrir el wireframe local wireframe_interactivo_sgr.html (100% offline). • Transiciones fluidas entre los integrantes sin pisarse el discurso.